

CMPUT 350 Lab 4 Prep Problems

Github Classroom: [Link](#)

Author: Daniel Cui

1 Introduction to Views, Cameras, and Image Zooming

Understanding how to manage coordinate systems and how to transform between them is necessary in order to implement graphics in interactive applications. In this lab, we'll explore how to implement a zoom feature for a resizable image viewing app using SFML, focusing on maintaining aspect ratios and computing coordinate transformations.

The application you'll build allows users to zoom into a "Where's Waldo" image by holding the spacebar, with the zoom centered at the mouse cursor position.

1.1 Coordinate Systems in Graphics

In SFML and most graphics systems, we work with multiple coordinate systems:

- **Window coordinates:** The physical pixel coordinates of the application window, which can change when the user resizes the window. Here, the top-left of the window is $(0, 0)$ and the bottom-right pixel is $(\text{window width} - 1, \text{window height} - 1)$. Notably, when you get your mouse position, they are expressed in these coordinates.
- **World coordinates:** A logical coordinate system that represents our scene, independent of the actual window size.
- **View coordinates:** The portion of the world that is currently visible, defined by a viewport and potentially transformed.

The relationship between these coordinate systems are defined in general by *transformations*. In SFML, `sf::View` is a convenient piece of machinery for handling conversions between window and world coordinates.

1.2 The View and Viewport System

Tutorial: [Link](#)

Documentation: [Link](#)

In SFML, a `sf::View` defines what portion of the world space is visible and where this space maps to the window space. The key components are:

- **Size:** A `sf::Vector2f`, the dimensions of the visible area in **world** coordinates
- **Center:** A `sf::Vector2f`, the center point of the view in **world** coordinates
- **Viewport:** A `sf::FloatRect`, the portion of the window where the view is rendered (in **normalized window** coordinates from 0.0 to 1.0)

You can make a `sf::Window` use a view via `window.setView(view)`. This means that from now on, the world-space rectangle centered at the view's center with its size is rendered into the window-space subrectangle given by the view's viewport.

1.3 Aspect Ratio and Letterboxing

When the window's aspect ratio doesn't match the world's aspect ratio, we need to handle this mismatch to prevent distortion. **Letterboxing** is a technique that maintains the correct aspect ratio by adding black bars (either horizontal or vertical) around the content.

For example, if we have a world view of 800×600 (4:3 ratio) and the window is resized to 1200×600 (2:1 ratio):

- The world should scale uniformly to fit within the window
- Black bars appear on the left and right sides (in this case, each of width 200 pixels)
- The viewport is adjusted to center the content

2 Overview

The `ZoomApp` loads embedded image data from `waldo_data.cpp` and automatically calculates a world size that maintains the aspect ratio of the original image, while maximally fitting into a given `maxWorldWidth` and `maxWorldHeight`.

It maintains two `Sf::Views`:

1. `mWorldViewDefault`, which has size equal to the world size, and centered at the center of the world. As given to you, its viewport on the window just stretches to the entire window. However, it ought to letterbox into the largest possible centered rectangle of the window that maintains its aspect ratio.
2. `mWorldViewZoomed`, which has size `mWorldSize / ZOOM_FACTOR`. You will be responsible for implementing `updateZoomView`, which sets the new world space position for this view given a mouse position. When zoomed in, this view needs to have the user's cursor point to the exact same thing the user was pointing at in the default view.

Once you have completed the problems, you will be able to zoom in like a magnifying glass on your cursor by pressing spacebar.

3 Problems

Complete the TODOs in `zoom.cpp`:

1. Implement Aspect Ratio Maintenance in `updateViewAfterResize()`

When the window is resized, uniformly scale and position the viewports of both views (`mWorldViewDefault` and `mWorldViewZoomed`) to maximally fit, centered within the window.

Reminder: The views' viewports use normalized coordinates (0.0 to 1.0), where (0,0) is the top-left corner and (1,1) is the bottom-right corner of the window.

2. Implement Zoom Transform Calculation in `updateZoomView()`

Given a mouse position, set the center of `mWorldViewZoomed` such that the zoomed-in view will have the user's cursor point to the same thing they were pointing to in the default view. High level steps are commented for you.

Hint: Determine the steps to go from the world-space desired center to the top-left of the zoomed-in world-space rectangle, then from this top-left world point to the thing you're pointing at (again in world space). Then solve the equation for the desired center.

3. Complete the Rendering in `render()`

Set the window's view to the appropriate view based on whether or not the user is zooming in.

3.1 Testing Your Implementation

Once you complete all functions, your zoom application should:

- Display the Waldo image centered in the window
- Maintain aspect ratio when the window is resized (with black letterbox bars)
- Zoom in 4× when spacebar is held, centered at the mouse position
- Return to normal view when spacebar is released
- Handle window resizing gracefully while maintaining the correct aspect ratio

Common Issues and Solutions:

- **Distorted image:** Check that viewport calculations maintain the aspect ratio
- **Zoom not centered on mouse:** Ensure proper coordinate conversion from pixel to world
- **Black screen:** Verify that the view and viewport are properly set
- **Zoom jumps:** Make sure zoom center is calculated only once when zoom starts

Footnotes

1. Here, `ZOOM_FACTOR = 4.f`. ↩